Pruebas de software: fundamentos, pruebas unitarias y E2E

Documento de apoyo académico • Octubre de 2026

Este material presenta los conceptos esenciales de las pruebas de software y algunas herramientas utilizadas en proyectos reales. Su propósito es servir como referencia durante el estudio de la calidad del software y la automatización de pruebas.

1. ¿Por qué probamos el software?

Probar software consiste en ejecutar verificaciones planificadas para encontrar defectos y obtener evidencia de que el sistema se comporta conforme a lo esperado. Las pruebas reducen riesgos, pero no demuestran la ausencia absoluta de errores. Una estrategia útil combina verificaciones rápidas de componentes con pruebas de flujos completos.

2. Niveles de prueba

Unitarias: Verifican una unidad pequeña de comportamiento, como una función o método, de forma aislada.

Integración: Verifican la interacción entre componentes, servicios, bases de datos o sistemas externos.

End-to-End (E2E): Recorren un proceso desde la perspectiva del usuario, por ejemplo iniciar sesión, registrar una solicitud y consultar su resultado.

Una prueba unitaria puede confirmar que una regla de descuento funciona. Una prueba E2E verifica que un usuario pueda realizar una compra desde la interfaz y observar su confirmación. Ambas aportan información distinta y son complementarias.

3. Pruebas unitarias con Java

JUnit 5 permite escribir y ejecutar pruebas automatizadas en Java. En una prueba se prepara una entrada, se invoca el comportamiento que se desea verificar y se compara el resultado obtenido con el esperado mediante una aserción.

```
public class CalculadoraDescuentos {
   public double aplicar(double precio, double porcentaje) {
      if (precio < 0 || porcentaje < 0 || porcentaje > 100)
         throw new IllegalArgumentException("Valores inválidos");
      return precio * (1 - porcentaje / 100);
   }
}
```

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraDescuentosTest {
   @Test
   void aplicaDescuentoDelDiezPorCiento() {
```

```
        var calculadora = new CalculadoraDescuentos();
        assertEquals(90000.0, calculadora.aplicar(100000, 10), 0.001);
    }

    @Test
    void rechazaPorcentajeFueraDeRango() {
        var calculadora = new CalculadoraDescuentos();
        assertThrows(IllegalArgumentException.class,
            () -> calculadora.aplicar(100000, 110));
    }
}
```

Conviene cubrir comportamientos válidos, entradas inválidas y casos límite. Una buena prueba tiene un nombre descriptivo, se puede ejecutar repetidamente y no depende del orden de ejecución de otras pruebas.

4. Pruebas End-to-End (E2E)

Las pruebas E2E automatizan recorridos representativos de un usuario a través de la aplicación. Pueden involucrar interfaz web, backend, autenticación y persistencia de datos. Son especialmente útiles para verificar procesos críticos, aunque suelen ser más lentas y sensibles a cambios de interfaz que las pruebas unitarias.

Ejemplo de recorrido: abrir la aplicación → iniciar sesión → registrar un pedido → confirmar el registro → verificar que el pedido aparece en la consulta.

5. Herramientas de automatización E2E

La siguiente tabla resume alternativas frecuentes. La selección depende del lenguaje del equipo, el tipo de aplicación, la infraestructura disponible y la facilidad de mantenimiento.

Herramienta	Lenguajes	Enfoque	Aspecto a considerar
Selenium WebDriver	Java, Python, C#, JavaScript	Web en varios navegadores	Amplio ecosistema; requiere diseñar esperas y estructura de pruebas.
Playwright	Java, TypeScript/JavaScript, Python, C#	Web moderna y mult navegador	Autoesperas, trazas y aislamiento de sesiones.
Cypress	JavaScript, TypeScript	Aplicaciones web	Experiencia de depuración y ejecución interactiva.
WebdriverIO	JavaScript, TypeScript	Web y móvil con integraciones	Ecosistema Node.js y múltiples servicios.
Robot Framework	Sintaxis tabular; librerías Python	Automatización por palabras clave	Legibilidad; depende de bibliotecas para controlar la interfaz.
Appium	Java, Python, JavaScript y otros	Aplicaciones móviles	Android e iOS; configuración de dispositivos.
Cucumber	Gherkin + lenguajes de implementación	Escenarios BDD	No controla el navegador por sí solo; se combina con Selenium u otro motor.
Karate	DSL sobre JVM	API y escenarios	Muy útil para servicios;

		integrados	automatización UI disponible.

6. Ejemplo básico de E2E con Selenium

El siguiente ejemplo utiliza un formulario público de demostración de Selenium. Requiere las dependencias de Selenium y JUnit 5 en Maven, así como Chrome instalado.

```
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import static org.junit.jupiter.api.Assertions.assertEquals;

class FormularioE2ETest {
   @Test
   void enviaFormulario() {
      WebDriver navegador = new ChromeDriver();
      try {
         navegador.get("https://www.selenium.dev/selenium/web/web-form.html");
         navegador.findElement(By.name("my-text")).sendKeys("Prueba E2E");
         navegador.findElement(By.cssSelector("button")).click();
         assertEquals("Received!",
            navegador.findElement(By.id("message")).getText());
      } finally {
         navegador.quit();
      }
   }
}
```

En proyectos más grandes se recomienda encapsular las acciones de cada pantalla con Page Object Model, usar identificadores estables y controlar las condiciones de espera. También es importante disponer de datos de prueba y entornos reproducibles.

7. ¿Cómo elegir una herramienta?

Para proyectos desarrollados principalmente en Java, Selenium + JUnit o Playwright para Java son alternativas razonables. Para equipos centrados en Angular, React y TypeScript, Playwright o Cypress suelen integrarse naturalmente. Si el objetivo principal es verificar contratos y flujos de API, pueden considerarse Karate o pruebas de integración con herramientas del propio backend. Para aplicaciones móviles nativas, Appium es una opción conocida.

8. Relación con el desarrollo profesional

Las pruebas automatizadas forman parte del ciclo de desarrollo: una historia de usuario define un comportamiento esperado, las pruebas unitarias protegen reglas de negocio, las pruebas de integración verifican colaboraciones y las E2E comprueban recorridos críticos. En CI/CD pueden ejecutarse automáticamente para detectar regresiones antes de publicar cambios.

9. Recursos de consulta

JUnit 5 — https://junit.org/junit5/docs/current/user-guide/

Selenium — https://www.selenium.dev/documentation/

Playwright Java — https://playwright.dev/java/

Cypress — https://docs.cypress.io/

Cucumber — https://cucumber.io/docs/

Karate — https://docs.karatelabs.io/

Appium — https://appium.io/docs/en/latest/

WebdriverIO — https://webdriver.io/

Robot Framework — https://robotframework.org/